

COP2034C Module 5 Programming Assignment

Instructions

In this assignment, we will continue building the Palmer Penguins program from Module 4. In that module, we added decision-making that allowed the user to select one penguin species and display information about that species. We also continued using parallel lists to keep related species data at matching indexes.

We will add loops so that the program can:

- use a **for** loop instead of repeating nearly identical output statements,
- allow the user to explore multiple species during the same program run,
- and count how many valid species the user explores.

We will practice:

- for loops,
- while loops,
- loop counters,
- enumerate(),
- combining loops with the branches created in Module 4.

As before, work incrementally: make one change, run the program, verify that it works, and then continue.

1. Start with Your Module 4 Solution

Open the Palmer Penguins notebook you completed for Module 4. Copy the notebook for Module 5. A suggested name is:

PalmerPenguinsM5.ipynb

The Python file you eventually download should have a meaningful name such as:

PalmerPenguinsM5.py

Do not rewrite the program from the beginning. Module 5 should build on your completed Module 4 program.

2. Update the ID Header

Update the ID header at the top of the program:

```
# PalmerPenguinsM5.py
# Your Name
# Submission Date
# Uses loops to repeatedly explore Palmer Penguin statistics.
```

Leave a blank line after the header.

3. Retain Your Module 4 Code

Keep the constants, lists, calculations, branches, and formatted output from Module 4. In particular, continue using the three parallel lists:

```
species_names
species_counts
species_percentages
```

The same index should continue to identify related information in all three lists. Module 4 already uses that arrangement to retrieve a selected species, count, and percentage.

4. Use a for Loop to Display the Species Statistics

Your Module 4 program contains separate output statements for Chinstrap, Gentoo, and Adelie:

```
Chinstrap: 68 (19.88%)
Gentoo: 123 (35.96%)
Adelie: 151 (44.15%)
```

Replace those repeated species-statistics statements with a **for loop**.

Use `enumerate()` so that each iteration provides both the current index and species name:

```
for index, species in enumerate(species_names):
```

Inside the loop, use the index to retrieve the matching values from:

```
species_counts
species_percentages
```

Each iteration should display one line in the same format used in Module 4. Do **not** hardcode the three species names, counts, or percentages inside the loop. When the loop finishes, the output should still be:

```
Chinstrap: 68 (19.88%)
Gentoo: 123 (35.96%)
Adelie: 151 (44.15%)
```

This change should affect how the output is generated, not what the user sees.

5. Add a Counter for Valid Species Selections

Before the interactive species-selection portion of the program, create a variable named “`valid_selections`” and initialize it to 0. This variable will count how many valid species the user explores during the program run. An invalid entry should not increase this counter.

6. Allow the User to Enter “Done”

7. Repeat Species Exploration with a while Loop

Module 4 processes one species selection and then ends. In Module 5, the program should continue asking the user for species until the user enters:

Done

Use a sentinel-controlled while loop. A useful overall pattern is:

```
species_choice = input(...)
while species_choice != "Done":
    # process the user's selection
    species_choice = input(...)
```

Move the existing Module 4 species-validation and species-processing code inside this loop. Your existing code that begins by checking:

```
if species_choice not in species_names:
```

should still determine whether the selection is valid. The branching logic that classifies and describes a valid species should also remain. Module 4 already performs the percentage classification, relative-position determination, and average comparison. The important difference is that the program should now return to the prompt after each selection.

8. Count Valid Selections

When the user enters a valid species, increase `valid_selections` by 1. For example, the update should occur somewhere inside the branch that processes a valid species.

If the user enters:

Gentoo
Emperor
Chinstrap
Done

then the `valid_selections` variable should finish with the value 2.

Emperor is invalid and should therefore not be counted.

9. Display a Final Summary

When the user enters Done, the while loop should end.

After the loop, display the number of valid species selections made during the program run.

For example:

You explored 2 valid species.
Exploration complete.

Use the `valid_selections` variable rather than placing a number directly in the output statement.

Expected Interactive Output

The first portion of the program should continue to display the Palmer Penguins dataset statistics from Module 4.

The species-statistics lines should look the same as before, even though they are now produced by a for loop.

The new interactive portion of a sample run might look similar to:

```
Explore a species:
```

```
Enter Chinstrap, Gentoo, Adelie, or Done: Gentoo
```

```
Gentoo has 123 observations, representing 35.96% of the dataset.  
Gentoo is classified as a medium species group.  
Gentoo is the middle-sized species group.  
Its observation count is above the dataset average of 114.0.
```

```
Enter Chinstrap, Gentoo, Adelie, or Done: Emperor
```

```
Emperor is not one of the species in this dataset.
```

```
Enter Chinstrap, Gentoo, Adelie, or Done: Chinstrap
```

```
Chinstrap has 68 observations, representing 19.88% of the dataset.  
Chinstrap is classified as a small species group.  
Chinstrap is the smallest species group.  
Its observation count is below the dataset average of 114.0.
```

```
Enter Chinstrap, Gentoo, Adelie, or Done: Done
```

```
You explored 2 valid species.  
Exploration complete.
```

Minor whitespace differences are acceptable as long as the output is readable and contains all required information.

Test Your Loops

Before submitting the assignment, test the program with several inputs during the same program run. At minimum, test this sequence:

```
Gentoo  
Emperor  
Chinstrap  
Done
```

Verify that:

- the program returns to the prompt after each entry,
- valid species display the correct statistics,
- invalid species display the Module 4 error message,
- the program stops when Done is entered,
- and the final valid-selection count is 2.

Also run the program once and enter Done immediately. The program should end normally and report 0 valid species selections.

Readability

Continue following the readability expectations from Module 4: meaningful variable names, consistent four-space indentation, reasonable line lengths, logical blank lines, and comments identifying major sections. Useful new comments might include:

```
# display species statistics using a loop
# initialize the valid selection counter
# repeatedly prompt the user to explore species
# update the number of valid selections
# display the exploration summary
```

Screenshot and Submission

Follow the same submission process used for Module 4. Run the completed program in Google Colab. For the screenshot, show enough of one execution to demonstrate that the program prompts for more than one species and reaches the final summary.

Suggested filenames:

- PalmerPenguinsM5.py
- PalmerPenguinsM5.png

Upload both files to your forked GitHub repository and submit the repository URL to Canvas. **Do not submit the .ipynb notebook.**